

Cuestionario modelos y servicios Web

1. Servidor web frente a servidor de aplicaciones

Apache HTTPD y Apache Tomcat ocupan capas distintas dentro de una infraestructura multicapa. Los dos atienden peticiones HTTP, pero lo hacen con propósitos diferentes.

	Apache HTTPD	Apache Tomcat
Función	Servidor web	Contenedor de servlets y JSP
Peticiones que procesa nativamente	Recursos estáticos (HTML, CSS, JavaScript, imágenes, vídeo, archivos) y rutas gestionadas por módulos	Peticiones que deben convertirse en una llamada a un servlet, JSP u otro componente Java
Funciones que asume	Terminación TLS, hosts virtuales, compresión, cachés, autenticación, cabeceras de seguridad, registros, control de acceso	Ciclo de vida de la aplicación, gestión de hilos, sesiones, despliegue de archivos WAR, componentes Jakarta Servlet y JSP
Relación con la base de datos	No accede	Accede; el pool de conexiones lo gestiona la aplicación o su framework

Tomcat no incorpora Apache. Ambos son programas independientes, y que la documentación de Tomcat lo denomine servidor web y de aplicaciones se debe a que también puede servir contenido estático, no a que integre el servidor web.

No es recomendable delegar el contenido estático de alto tráfico en el motor de aplicaciones porque cada petición así obliga a la JVM a activar un hilo, consumir memoria de heap y atravesar el contenedor de servlets, con un coste por unidad de recurso muy superior al de una lectura de archivo en el sistema de ficheros. El servidor web resuelve además ese tráfico con herramientas que el contenedor no ofrece: compresión, rangos de bytes, caché en memoria, negociación de codificación y control de límites de peticiones. En una infraestructura de producción, el servidor frontal sirve los estáticos y transmite al contenedor de aplicaciones únicamente las peticiones que requieren lógica de negocio.

2. Renderizado y ejecución en ausencia de servidor HTTP

Es viable visualizar una página sin que medie un servidor web cuando se trata de un documento HTML estático con recursos referenciados por rutas locales. Al abrir `file:///C:/proyecto/index.html`, el navegador no realiza ninguna petición HTTP: resuelve el URI como una ruta del sistema de ficheros, lee el documento desde el disco, interpreta el árbol HTML y carga la hoja de estilos, las imágenes y los scripts clásicos mediante referencias locales. El resultado es una representación correcta del documento.

Ese contexto presenta restricciones de seguridad relevantes frente al uso de peticiones asíncronas, CORS y módulos ECMAScript:

- **Origen opaco.** El origen de un documento `file://` es opaco, representado como `null`. La política de mismo origen trata ese valor como distinto de cualquier otro, de modo que el recurso no puede participar en decisiones de compartición.
- **Peticiones asíncronas.** `fetch()` y `XMLHttpRequest` resuelven su respuesta aplicando las reglas de CORS contra un origen `null`. El resultado habitual es que la solicitud se rechaza con un error de

CORS antes de llegar al servidor, con independencia de que se encuentre en el mismo directorio.

- **Módulos ECMAScript.** Un `<script type="module" src="app.js">` no puede ejecutarse desde el sistema de archivos. La especificación de módulos exige que el recurso se obtenga mediante un mecanismo de red, y los navegadores rejeitan la carga local. La razón es que un módulo comparte ámbito con el resto del código, de modo que se le aplica un modelo de seguridad más estricto que al script clásico.
- **Tipos MIME.** Los navegadores verifican que un módulo se sirva con un tipo MIME de JavaScript válido; los esquemas locales no siempre satisfacen esa comprobación.

Para reproducir las condiciones de una aplicación servida por HTTP, con origen real, CORS verificable y módulos operativos, hay que utilizar un servidor local:

```
| python -m http.server 8000
```

y abrir `http://localhost:8000/`. En ese contexto el documento, los módulos y la API se sirven por HTTP, el origen es identificable y la política de mismo origen se aplica con normalidad.

3. Transición y obsolescencia tecnológica

Las tecnologías históricas analizadas cumplieron su función en un contexto concreto, pero presentan debilidades de seguridad, consumo de recursos e incompatibilidad multiplataforma.

Tecnología	Debilidades	Estándares y alternativas vigentes
Scripts CGI en Perl y C	El modelo clásico creaba un proceso independiente por petición, con el coste asociado de crear y destruir procesos de forma constante. Una implementación mal diseñada podía además con inyección de comandos, exposición de variables de entorno, validación insuficiente de entrada y agotamiento de recursos.	CGI permanece como especificación abierta (RFC 3875). Sus funciones se cubren con variantes persistentes que mantienen el proceso vivo: FastCGI, SCGI, PHP-FPM; en el lado servidor, Servlets y Jakarta EE; en el cliente, interfaces Fetch y XMLHttpRequest.
Applets de Java	Requerían un complemento y una máquina virtual instalada en el equipo del cliente. El complemento ampliaba la superficie de ataque, exigía firma digital o certificados de firma específicos y su integración difería entre navegadores. Consumían memoria y CPU del equipo del usuario.	La API Applet se marcó para eliminación (JEP 398). Sus funciones se cubren en el cliente con HTML5, JavaScript, WebAssembly, Web Workers y las APIs de la plataforma web; en el servidor, con Java bajo un contenedor de servlets.
Controles ActiveX	Ligados al sistema operativo Windows y al navegador Internet Explorer, con acceso a recursos del sistema y una superficie de seguridad considerable. No operaban en otros sistemas.	Funciones equivalentes mediante HTML, CSS, JavaScript y WebAssembly. Las extensiones de navegador siguen existiendo como tecnología abierta, pero con aislamiento de procesos, permisos declarados y revisión de seguridad.
VBScript	Diseñado para el ecosistema de Microsoft e Internet Explorer; nunca fue un estándar abierto ni multiplataforma.	ECMAScript como estándar abierto del lado cliente, con el documento HTML5 como formato. En el servidor, implementaciones de .NET, Node.js, PHP, Python y Java.

4. Comparativa de arquitecturas base (LAMP frente a WISA) y aprovisionamiento

LAMP y WISA son combinaciones de dos opciones que cubren capas distintas de una aplicación web. La diferencia relevante no es técnica sino organizativa y de licenciamiento.

	LAMP	WISA
Sistema operativo	Linux	Windows
Servidor web	Apache HTTP Server	Internet Information Services
Base de datos	MySQL, aunque admite MariaDB, PostgreSQL y otros motores	SQL Server
Lenguaje de servidor	PHP, aunque admite Perl y Python	ASP y ASP.NET
Modelo de licencia	Linux, Apache y PHP se distribuyen con licencias abiertas. MySQL Community también es libre, pero algunas ediciones comerciales requieren pago	IIS y SQL Server son propietarios y requieren licencia. Parte de la pila .NET es de código abierto
Coste total	Sin coste de licencia, con coste de servidor, mantenimiento, seguridad y copias de seguridad	Coste de licencia de Windows y SQL Server, además del coste de operación y del conocimiento del ecosistema

El aprovisionamiento manual de estas pilas sobre el sistema anfitrión implicaba un proceso repetido en cada máquina, con las siguientes dificultades de mantenimiento:

1. Instalación del sistema operativo y aplicación de actualizaciones.
2. Instalación de versiones concretas de Apache, PHP, MySQL y sus librerías.
3. Creación de usuarios, permisos, certificados y reglas de filtrado.
4. Configuración del arranque del servicio y de la rotación de registros.
5. Repetición íntegra del procedimiento en cada servidor nuevo del crecimiento.
6. Sincronización manual de versiones y resolución de divergencias entre entornos.

El resultado es deriva de configuración, conflictos de puertos, dependencias implícitas, actualizaciones incompletas y dificultad para reconstruir un servidor. Las máquinas virtuales aíslan el sistema completo, pero cada una incorpora un sistema operativo entero y reserva recursos por sí sola.

Los contenedores resuelven la reproducibilidad de esa fase. Una imagen de Docker empaqueta la aplicación, sus dependencias y su configuración, y un contenedor se ejecuta como proceso aislado que comparte el núcleo del anfitrión. Un `Dockerfile` documenta la construcción de la imagen y Docker Compose declara servicios, redes, volúmenes y variables de entorno. Un orquestador puede crear réplicas, sustituir instancias defectuosas y distribuir la carga, de modo que el aprovisionamiento pasa a ser declarativo y reproducible en lugar de manual.

5. Alojamientos múltiples y Virtual Hosts

El fichero `000-default` corresponde a la distribución de Apache en Debian y Ubuntu, donde se ubica en `/etc/apache2/sites-available/000-default.conf` y se habilita mediante un enlace simbólico desde `sites-enabled`. Las directivas que intervienen tienen responsabilidades distintas:

Directiva	Función
Listen 80 y Listen 443	Determinan los sockets en los que Apache acepta conexiones
DocumentRoot	Directorio del sistema de ficheros desde el que se sirve el sitio
ServerName	Nombre principal del alojamiento virtual
ServerAlias	Nombres adicionales asociados al mismo alojamiento
<VirtualHost *:80>	Declara un alojamiento que atiende cualquier dirección local en el puerto 80

```

<VirtualHost *:80>
    ServerName ejemplo.local
    ServerAlias www.ejemplo.local
    DocumentRoot /var/www/ejemplo
</VirtualHost>

<VirtualHost *:80>
    ServerName tienda.local
    DocumentRoot /var/www/tienda
</VirtualHost>

```

El mecanismo de discriminación se apoya en la cabecera `Host`. La dirección IP y el puerto identifican el servidor, pero no el sitio: un único Apache escucha en la misma dirección IPv4 y en los puertos 80 y 443 para todos los alojamientos. Al recibir una petición, el servidor combina la dirección de destino, el puerto de destino y el valor de la cabecera `Host`, y selecciona el alojamiento virtual cuyo `ServerName` o `ServerAlias` coincide con ese valor. La resolución de nombre debe apuntar todos los dominios a la misma dirección para que las peticiones lleguen al servidor. Si el nombre recibido no coincide con ningún alojamiento, se atiende el primer alojamiento que coincida con dirección y puerto.

En HTTPS existe una limitación que condiciona el procedimiento: el servidor debe conocer qué certificado presentar antes de poder leer la cabecera `Host`, porque la petición aún no se ha descifrado. Para resolverlo se emplea la extensión Server Name Indication (SNI), que incluye el nombre solicitado en el mensaje `ClientHello` del inicio de TLS. El servidor selecciona certificado y alojamiento en esa fase, y una vez completado el cifrado vuelve a utilizar `Host` y las directivas del alojamiento para atender la petición.

6. Dimensionamiento: escalabilidad vertical frente a horizontal

Criterio	Escalabilidad vertical (scale-up)	Escalabilidad horizontal (scale-out)
Ventajas	Implantación y monitorización sencillas. No requiere modificar la aplicación. Adecuada para servicios que no pueden duplicarse, como la base de datos	Crecimiento gradual y mejora de la disponibilidad cuando se añaden mecanismos de conmutación. Permite aprovechar la capacidad por módulos
Limitaciones técnicas	Existe un techo físico: alcanzado, exige migración a otro nodo. Aumentar un componente no resuelve un cuello de botella situado en otro, como el acceso a disco	Exige balanceo, coordinación y comunicación entre nodos, estrategia coherente de sesión, despliegue automatizado y una base de datos capaz de sostener la concurrencia. La latencia de red reduce el rendimiento
Costes	Puede requerir parada para cambiar hardware, con un incremento notable del precio al aproximarse al límite	Aumenta el coste por nodos adicionales, conmutación, almacenamiento compartido y servicios de orquestación, si bien permite crecer por fases y reutilizar equipos pequeños

Criterio	Escalabilidad vertical (scale-up)	Escalabilidad horizontal (scale-out)
Puntos únicos de fallo	El nodo principal es un punto único de fallo: su caída supone la pérdida de disponibilidad	La sustitución de un nodo reduce el riesgo, pero el balanceador, la base de datos, la red y el almacenamiento siguen siendo puntos únicos si no se redundan
Sesiones y estado	El estado se mantiene en memoria local con facilidad	Cada nodo solo conserva parte del estado, por lo que se requiere afinidad, replicación o almacenamiento compartido

Ambos criterios son complementarios. El escalado vertical suele ser el recurso inicial más sencillo y económico, mientras que el horizontal aporta flexibilidad y disponibilidad cuando la demanda es variable o elevada.

7. Persistencia de sesiones en entornos distribuidos

HTTP es un protocolo sin estado, de modo que identificar al usuario entre peticiones exige un mecanismo de sesión. Al balancear el tráfico entre servidores idénticos aparece el problema: la segunda petición del usuario puede recibirla un nodo que no conserva esa sesión en su memoria local. Existen tres enfoques.

Sticky sessions. Mantienen la afinidad entre el usuario y un nodo concreto, por dirección IP o mediante cookie que indica el nodo que debe atender la siguiente petición. Su principal ventaja es la implantación simple, ya que la aplicación conserva sesiones locales sin replicación. Presentan el inconveniente de un reparto de carga desequilibrado cuando un usuario genera muchas peticiones, la pérdida de afinidad cuando el cliente cambia de dirección o atraviesa otra red de proxies, y la pérdida de la sesión si el nodo asignado falla.

Replicación en clúster. Tomcat y otros servidores en clúster replican los objetos de sesión entre nodos, de forma que cualquier nodo puede atender la petición. El coste es de CPU, red y memoria, con la latencia derivada; además exige serializar y validar los objetos, controlar la concurrencia y arbitrar el caso en que dos peticiones modifiquen la misma sesión. Una base de datos compartida evita la réplica pero introduce un punto de concentración si cada lectura o escritura de sesión pasa por ella.

Aplicaciones sin estado. El nodo no conserva la sesión en memoria. La sesión reside en un almacén distribuido como Redis o Memcached, identificado por un token opaco que cada nodo consulta, o bien en un token firmado (JWT) que transporta la información de sesión o autorización y que los nodos validan por firma, sin consultar a ningún otro nodo. Redis es la opción más habitual por su caducidad nativa. Un JWT no es seguro por defecto: requiere algoritmos criptográficamente fuertes, control de caducidad, protección de la clave, exclusión de datos sensibles y un mecanismo de revocación o rotación. Si la sesión viaja en cookie, debe transportarse por HTTPS con los atributos `HttpOnly`, `Secure` y `SameSite`.

8. Algoritmos de reparto de carga

Un balanceador de capa 7 puede tomar la decisión a partir de la URL, las cabeceras o la cookie de sesión, y no solo de la conexión. Los algoritmos habituales son los siguientes.

Round Robin. Distribuye las peticiones de forma secuencial entre los nodos activos, siguiendo el ciclo A, B, C, A, B, C. Es el algoritmo más sencillo y no requiere estado ni seguimiento. La carga queda

equilibrada únicamente si todas las peticiones tienen un coste similar; ante peticiones con duraciones muy distintas, como una consulta lenta frente a un recurso estático en caché, la rotación ignora esa diferencia y produce desequilibrio.

LRU (Least Recently Used). Es un criterio basado en la recencia de uso, empleado en las cachés para decidir qué elemento se descarta: se retira el que lleva más tiempo sin ser solicitado, lo que mantiene disponible el conjunto de datos de uso más reciente. Aplicado a la distribución de tráfico, el criterio equivalente prioriza las rutas o los recursos solicitados más recientemente, de modo que el nodo o la entrada de caché asociada a ellos conserva la prioridad. Su comportamiento depende del histórico de peticiones y se adapta mejor a patrones con accesos concentrados que Round Robin, aunque concentra la carga en las entradas más demandadas.

Least Connections. Selecciona el nodo con menor número de conexiones activas en ese momento. La decisión se basa en el estado real y no en un contador acumulado, por lo que resulta más equilibrado cuando conviven peticiones de duración dispar. Su fiabilidad depende de que el balanceador contabilice correctamente las conexiones activas y persistentes, y no considera directamente el consumo de CPU o memoria.

Balanceo ponderado por capacidad de nodo. Asigna a cada nodo un peso que refleja su capacidad y reparte las peticiones en proporción. Permite aprovechar correctamente el hardware heterogéneo, como un nodo de ocho núcleos junto a otro de dos, y ajustar la capacidad de forma progresiva. Si los pesos no se mantienen alineados con la capacidad real, la consecuencia es saturación de un nodo o desperdicio de otro.

9. Patrón proxy inverso y protocolo AJP

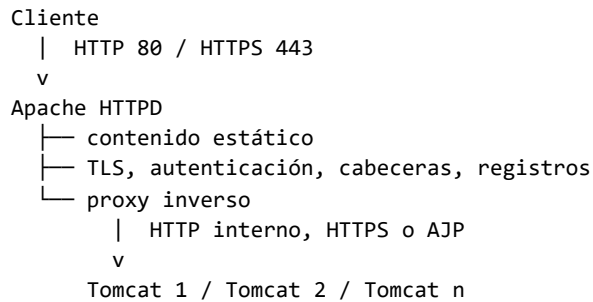
En el patrón de proxy inverso Apache HTTPD ocupa la posición frontal y Tomcat queda detrás. La vinculación se configura con ProxyPass, que establece el reenvío, y BalancerMember, que declara cada nodo del conjunto. ProxyPreserveHost On hace que Tomcat reciba el nombre de host original, necesario para construir redirecciones y enlaces absolutos correctos.

AJP/1.3 es un protocolo binario diseñado para la integración de Tomcat con un servidor web frontal, y escucha en el puerto 8009. El conector HTTP de Tomcat utiliza por defecto el 8080. AJP no es HTTP: transmite atributos internos de la petición, lo que amplía la superficie de ataque y condiciona la configuración de seguridad.

La colocación de un servidor web frontal en lugar de exponer el 8080 a redes públicas se justifica por tres motivos:

- **Seguridad.** El servidor frontal centraliza autenticación, autorización, cabeceras de seguridad, límites de peticiones y filtrado. Eludirlo deja el contenedor de aplicaciones accesible directamente, y en el caso de AJP requiere además un secret no vacío, secretRequired=true y una interfaz de escucha restringida a la red privada o al bucle local.
- **Descarga de SSL/TLS.** Terminar TLS en el servidor frontal evita desplegar el módulo criptográfico en la JVM, reduce el consumo de recursos del contenedor y permite a este atender el tráfico interno en claro por una red de confianza. Si la red no es confiable, esa variante debe sustituirse por un proxy HTTP con TLS.
- **Administración de puertos.** Exponer únicamente el 80 y el 443 hacia el exterior mantiene cerrados el 8080 y el 8009. El balanceador puede además comprobar el estado de cada nodo antes

de enviarle tráfico, ocultar la estructura interna de la aplicación y conmutar de nodo sin modificar las URLs públicas. Los endpoints de administración del balanceador deben quedar protegidos con autenticación, autorización y restricción de red.



10. Evolución en la administración de servicios en GNU/Linux

Las guías históricas hacen uso de scripts SysV en `/etc/init.d/apache2` y de `apachectl` sobre distribuciones antiguas. Las distribuciones Linux actuales han adoptado **systemd** como gestor de sistema e inicio, y las guías de Apache asumen esta transición.

La unidad se denomina `apache2.service` en Debian y Ubuntu, y `httpd.service` en RHEL, CentOS Stream, Rocky Linux y AlmaLinux. La sintaxis necesaria es la siguiente:

Objetivo	Debian y Ubuntu	RHEL y derivados
Iniciar	<code>sudo systemctl start apache2</code>	<code>sudo systemctl start httpd</code>
Reiniciar	<code>sudo systemctl restart apache2</code>	<code>sudo systemctl restart httpd</code>
Detener	<code>sudo systemctl stop apache2</code>	<code>sudo systemctl stop httpd</code>
Comprobar estado detallado	<code>sudo systemctl status apache2 --no-pager --full</code>	<code>sudo systemctl status httpd --no-pager --full</code>
Habilitar arranque automático	<code>sudo systemctl enable apache2</code>	<code>sudo systemctl enable httpd</code>
Deshabilitar arranque automático	<code>sudo systemctl disable apache2</code>	<code>sudo systemctl disable httpd</code>
Consultar estado	<code>sudo systemctl is-active apache2</code>	<code>sudo systemctl is-active httpd</code>
Consultar arranque automático	<code>sudo systemctl is-enabled apache2</code>	<code>sudo systemctl is-enabled httpd</code>

`status` muestra el estado de la unidad, el proceso principal, los registros recientes y los errores. Antes de recargar conviene validar la sintaxis de la configuración, que en Debian y Ubuntu se comprueba con `sudo apache2ctl configtest` y en RHEL con `sudo apachectl configtest`. Si la respuesta es `Syntax OK`, el servicio puede recargarse. `daemon-reload` solo es necesario tras crear o modificar una unidad de `systemd`.

Fuentes consultadas

- Apache HTTP Server 2.4. Documentación principal. <https://httpd.apache.org/docs/2.4/en/>

- Apache HTTP Server 2.4. Virtual hosts basados en nombre. <https://httpd.apache.org/docs/2.4/en/vhosts/name-based.html>
- Apache HTTP Server 2.4. Core module, directivas Listen, DocumentRoot y ServerName. <https://httpd.apache.org/docs/2.4/en/mod/core.html>
- Apache HTTP Server 2.4. mod_proxy, ProxyPass y consideraciones de control de acceso. https://httpd.apache.org/docs/2.4/en/mod/mod_proxy.html
- Apache HTTP Server 2.4. mod_proxy_balancer, algoritmos de reparto. https://httpd.apache.org/docs/2.4/en/mod/mod_proxy_balancer.html
- Apache HTTP Server 2.4. mod_proxy_ajp. https://httpd.apache.org/docs/2.4/en/mod/mod_proxy_ajp.html
- Apache Tomcat 11. Documentación principal. <https://tomcat.apache.org/tomcat-11.0-doc/index.html>
- Apache Tomcat 11. Conector AJP/1.3, puerto 8009 y requisitos de seguridad. <https://tomcat.apache.org/tomcat-11.0-doc/config/ajp.html>
- Apache Tomcat 11. Clustering y replicación de sesiones. <https://tomcat.apache.org/tomcat-11.0-doc/config/cluster.html>
- Docker. Qué es un contenedor. <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>
- MDN Web Docs. CORS cuando la URL no es HTTP. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS/Errors/CORSRequestNotHttp>
- MDN Web Docs. Módulos de JavaScript. <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules>
- OWASP. Session Management Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- Redis. Session store. <https://redis.io/docs/latest/develop/use-cases/session-store/>
- IETF. RFC 3875, The Common Gateway Interface. <https://www.rfc-editor.org/rfc/rfc3875>
- OpenJDK. JEP 398, Deprecate the Applet API for Removal. <https://openjdk.org/jeps/398>
- systemd. Manual de systemctl. <https://freedesktop.org/software/systemd/man/latest/systemctl.html>